

What We Need to Build

Problem Statement → Context Layer Track

CORE REQUIREMENTS

- Choose 1 platform use case — e.g. hackathon, food delivery, coding, freelance.
- Define “User Context” for that platform.
- Collect platform activity / behavior and external data like LinkedIn / GitHub only if relevant.
- Build a Context Layer above the raw database: combine + structure user signals and generate a rich user profile, not just raw fields.
- Build a Conversational AI Agent that can check if a person exists, answer questions about them, and synthesize information from all available context.

WHAT MAKES IT STAND OUT

- Generalizable architecture → works beyond the chosen use case.
- Accurate, useful context → not a data dump.
- Bonus: Build matchmaking / recommendations using context.
- Bonus: Any other useful application powered by the Context Layer.

**Our interpretation: build understanding,
not another data dump.**

Facts are not Context

Platforms already know a lot. The missing layer is understanding what those facts mean together.

RAW FACTS

Name	Aashutosh
Account age	190 days
Plan	Pro
Feature A used	500 times
Feature B used	0 times
Errors today	10
Usually	0 errors
Teammates	20
Support tickets	2

Those are facts.



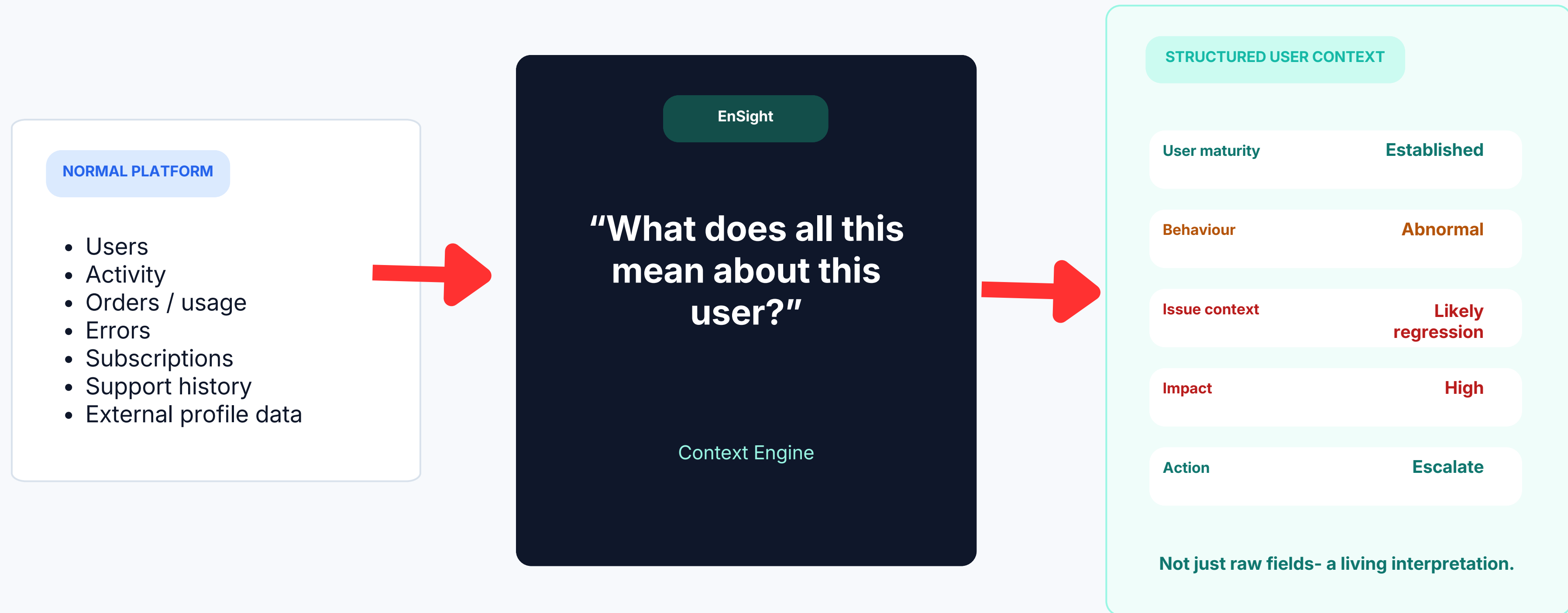
SYNTHESIZED CONTEXT

- 1 This is an established, high-usage customer.
- 2 Their behaviour today is highly abnormal.
- 3 The problem is probably not onboarding.
- 4 This issue is affecting a core workflow.
- 5 This deserves urgent attention.

That second object is context.

So what are WE building?

Raw user data → meaningful user understanding



Why not another RAG system?

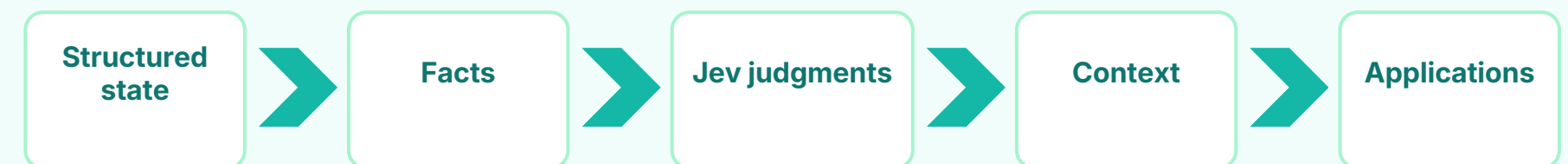
Why not chunking → embedding → retrieval?

COMMON APPROACH



We are ultimately building on top of an external application. We already have structured platform data, so we use that structure to our advantage and get insights as quickly as we can.

ENSIGHT APPROACH



Keep structured state structured end-to-end.

EXAMPLE

```
{
  "account_age": 190,
  "normal_error_rate": 0.003,
  "current_error_rate": 0.87,
  "configuration_changed": false,
  "daily_usage": 120000
}
```

Postgres can tell you

current_error_rate = 87%

Your application actually wants

BUT

- **This is an established customer.**
- **Their current behaviour is extremely abnormal.**
- **There is no evidence that they changed their configuration.**
- **This looks substantially more like a platform-side problem than a beginner configuration issue.**
- **Impact is probably high.**

JSONB is not the innovation. The Context Engine that produces the JSONB is the innovation.

New kid in the market: Jev

A machine-native decision model for fast, typed judgments on software state.

Instead of generating conversational text or code paragraphs, Jev's strict role is to take a piece of software state (like an email, log line, support ticket, or JSON blob) along with typed questions, and immediately output precise, typed answers, such as boolean values, category selections, or rubric scores.

Key Performance Capabilities

- Blazing Speed** 70–500 ms; roughly 40–200× faster than frontier LLMs on evaluation tasks.
- Economic Advantage** \$0.042 / 1M input tokens; output tokens free.
- No Structural Errors** Strict type safety → predictable structured outputs instead of formatting failures.

WHY IT FITS ENSIGHT

STATE

{ JSON }



Jev

Choice
Score
Yes / No



**Typed
decisions**

Fast enough to be part of a live context pipeline ,not only a chatbot call.

[1][2][3] Source references preserved from the original draft.

How it matters for us

One responsibility per layer — no model doing work that code already does better.

FACTS

Database / code answers facts

- How old is this account?
- How many requests were made?
- Did the error rate increase?
- Which features do they use?

JUDGMENT

Jev interprets those facts

- Is this user a mature user?
- Is this behaviour anomalous?
- Onboarding friction or platform problem?
- How severe does this situation appear?

LANGUAGE

LLM explains it to a person

- "Tell me what's going on with this customer."
- Summarize the stored context.
- Explain the evidence clearly.
- Only generate language when needed.

SQL handles facts. Jev handles judgment. The LLM handles language.

Applications of EnSight

- Support Intelligence
- Marketing Intelligence
- Product Intelligence

For Today

- Context-aware Support

Same message. Different context.

EnSight → Context-aware Support

USER A



"The product isn't working."

User A

Joined yesterday
Barely used product
Setup incomplete
No historical baseline

Context:

Likely new-user setup friction
Low impact
Normal support path

Action

Do not escalate

USER B



"The product isn't working."

User B

Customer for 2 years
Large workspace
Uses product every day
Historically stable
Sudden failures today
No configuration changes

Context:

Major behavioural anomaly
Likely genuine product problem
High impact
Priority escalation

Action

Escalate immediately



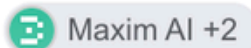
A traditional ticket sees the same sentence twice. EnSight understands two completely different situations.

Why EnSight?

A living context layer that humans, agents, and applications can directly consume.

- First class compatibility with any application
- Scalable
- Privacy / Compliant : self-host jev and LLM, no giant infra for inference
- Dirt Cheap
- flexible usecases

Google Gemini 2.5 Flash-Lite

- **Input Price:** \$0.10 per 1 million tokens
- **Output Price:** \$0.40 per 1 million tokens
- **Context Window:** ~1 million tokens
- **Type:** Traditional multimodal lightweight chat/text model 

TypeSafe AI Jev (Jev 1 / 1.13)

- **Input Price:** \$0.042 per 1 million tokens (\$42 per billion tokens)
- **Output Price:** Free (\$0.00) — output tokens are structured decisions ("too cheap to meter")
- **Context Window:** Up to 64k tokens per request (32k state + question)
- **Type:** Specialized "System One" decision and classification model (does not generate free-form text) 

OUR CORE IDEA

**Platforms don't have a data problem.
They have a context problem.**

EnSight continuously converts fragmented signals into a structured, living understanding of each user.

Understand first. Ask later.